Sede Regional del Pacífico

Bachillerato en Informática Empresarial

Curso IF-5100 Administración de Bases de Datos

Investigación:

Tema: Control de Concurrencia: Bloqueos y Deadlocks

Docente:

Josué Quesada

Estudiantes:

Ana Pérez Ríos B95925

Madelyn Medina Hernández C14601

Leandro Barrantes Solórzano B90975

I Ciclo 2026

Introducción

En el ámbito de las bases de datos, uno de los principales retos es garantizar que múltiples usuarios puedan acceder y modificar la información de manera simultánea sin comprometer la integridad de los datos. Este fenómeno, conocido como concurrencia, es fundamental en sistemas modernos donde varias transacciones ocurren al mismo tiempo, como en sistemas bancarios, plataformas de comercio electrónico y aplicaciones empresariales, en los cuales la disponibilidad y consistencia de la información resultan críticas.

En esta investigación se abordarán los conceptos clave relacionados con la concurrencia en bases de datos, así como los problemas que pueden surgir, debido a que el acceso concurrente a los mismos datos puede generar inconsistencias, pérdida de información o resultados incorrectos si no se gestiona adecuadamente. Asimismo, se expondrá acerca de las herramientas utilizadas para su control, haciendo énfasis en los bloqueos, los cuales permiten restringir el acceso a los datos cuando una transacción los está utilizando, y en los deadlocks, que se encargan de detectar y resolver situaciones en las que dos o más transacciones quedan en espera indefinida por recursos compartidos.

Asimismo, se analizará el papel de las transacciones dentro del control de concurrencia, ya que estas permiten agrupar operaciones en unidades lógicas que cumplen con propiedades que garantizan la integridad de los datos, incluso en situaciones de fallo o acceso simultáneo, asegurando así un comportamiento confiable del sistema.

De esta manera, se busca comprender la importancia de estos mecanismos en la gestión eficiente, segura y confiable de la información en sistemas modernos, destacando su impacto en el correcto funcionamiento de los sistemas de bases de datos en entornos multiusuario.

Objetivo general

Analizar el control de concurrencia en bases de datos, mediante el estudio de los mecanismos de bloqueos y la gestión de deadlocks, para comprender su importancia en la integridad y consistencia de la información.

Objetivos específicos

- Explicar el concepto de concurrencia en bases de datos, mediante el análisis de entornos multiusuario, para comprender su importancia en el acceso simultáneo a la información.
- Identificar los principales problemas de concurrencia, mediante el estudio de inconsistencias y conflictos entre transacciones, para analizar los riesgos en la gestión de datos.
- Analizar los mecanismos de control de concurrencia, especialmente los bloqueos y los niveles de aislamiento, mediante la revisión de su funcionamiento, tipos, compatibilidad, ventajas y desventajas, para evaluar su impacto en la integridad y el rendimiento de la base de datos.
- Comprender el fenómeno de los deadlocks, mediante el estudio de sus causas, condiciones, técnicas de detección, prevención y solución, para evitar interrupciones en los sistemas de bases de datos

Desarrollo

Concurrencia

La concurrencia en bases de datos se refiere a la capacidad de un sistema para permitir que múltiples usuarios o procesos accedan y modifiquen datos de manera simultánea. Este concepto es esencial en entornos multiusuario, donde varias transacciones pueden ejecutarse de forma paralela.

Según Silberschatz, Korth y Sudarshan (2011), la concurrencia permite mejorar la eficiencia del sistema al aprovechar mejor los recursos disponibles, pero requiere mecanismos de control para evitar inconsistencias en los datos.

De igual manera, Elmasri y Navathe (2016) indican que la ejecución concurrente de transacciones debe producir resultados equivalentes a una ejecución secuencial, lo que se conoce como serializabilidad, siendo este uno de los objetivos principales del control de concurrencia.

Sin embargo, aunque la concurrencia mejora el rendimiento, también introduce riesgos, ya que el acceso simultáneo a los mismos datos puede generar inconsistencias si no se controla adecuadamente. Por ello, es necesario implementar mecanismos de control de concurrencia que regulen el acceso a los datos.

Problemas de concurrencia

Cuando no se implementan mecanismos adecuados de control, la concurrencia puede generar diversos problemas que afectan la consistencia de los datos.

Uno de los más comunes es la actualización perdida, que ocurre cuando dos transacciones modifican el mismo dato y una sobrescribe el resultado de la otra. También se presentan las lecturas sucias, en las que una transacción accede a datos no confirmados por otra, lo que puede provocar inconsistencias si estos cambios se revierten posteriormente.

Otro problema es la lectura no repetible, que sucede cuando una misma consulta produce resultados diferentes dentro de una misma transacción debido a modificaciones externas. Asimismo, la lectura fantasma ocurre cuando aparecen nuevos registros en una consulta repetida.

Según Thomas Connolly y Carolyn Begg (2015), estos problemas son consecuencia directa del acceso concurrente sin control y pueden comprometer la confiabilidad del sistema si no se gestionan adecuadamente

Por esta razón, el control de concurrencia es esencial para garantizar que la ejecución concurrente de transacciones produzca resultados equivalentes a una ejecución secuencial (propiedad de serializabilidad).

Transacciones

Una transacción es un conjunto de operaciones que se ejecutan como una sola unidad de trabajo y que agrupa una o varias operaciones sobre la base de datos. Su objetivo es garantizar que las operaciones se ejecuten de forma completa y correcta. Si ocurre un error, todos los cambios pueden revertirse para mantener la consistencia.

Las transacciones cumplen con las propiedades ACID, fundamentales para asegurar la confiabilidad del sistema:

- Atomicidad: la transacción se ejecuta completamente o no se ejecuta.
- Consistencia: la base de datos pasa de un estado válido a otro estado válido.
- Aislamiento: las transacciones no interfieren entre sí.
- Durabilidad: los cambios permanecen incluso después de fallos.

De acuerdo con Silberschatz et al. (2011), estas propiedades son esenciales para mantener la integridad de los datos en entornos concurrentes.

Asimismo, Elmasri y Navathe (2016) destacan que las transacciones permiten coordinar múltiples operaciones, reduciendo los errores derivados de accesos simultáneos.

Además, las transacciones son la base del control de concurrencia, ya que permiten manejar múltiples operaciones sin perder integridad.

Ventajas del control de concurrencia

El control de concurrencia permite gestionar el acceso simultáneo a los datos, garantizando la integridad y consistencia de la información. Su implementación aporta múltiples ventajas para el buen funcionamiento de la base de datos. Sus principales ventajas son:

- En primer lugar, asegura la integridad de los datos, ya que permite evitar inconsistencias provocadas por accesos simultáneos, asegurando que la información se mantenga correcta y confiable.
- Garantiza el aislamiento de las transacciones, permitiendo que múltiples usuarios trabajen sin interferencias y cumpliendo con la propiedad de aislamiento del modelo ACID .
- Mejora el rendimiento del sistema, ya que permite ejecutar varias transacciones al mismo tiempo, optimizando el uso de los recursos.
- Contribuye a mantener la consistencia en entornos multiusuario, Asegura que, aunque varias transacciones se ejecuten al mismo tiempo, el resultado final sea coherente y equivalente a una ejecución secuencial .
- Reduce problemas como actualizaciones perdidas, lecturas sucias y otros conflictos derivados de la concurrencia.

Según Thomas Connolly y Carolyn Begg (2015), el control de concurrencia es fundamental para garantizar que las bases de datos funcionen de manera eficiente y confiable en sistemas modernos.

Bloqueos

¿Qué es un bloqueo?

Dado lo anterior mencionado, se entiende como concurrencia en bases de datos la garantía de la integridad de los datos ante accesos simultáneos de múltiples transacciones. En este sentido, para restringir dichos accesos existe el uso de bloqueos (locks), los cuales permiten restringir accesos conflictivos a recursos como filas, páginas o tablas, asegurando la consistencia de la información (IBM Corporation, 2018).

Bajo este enfoque, un bloqueo se interpreta como el mecanismo mediante el cual el sistema de gestión de bases de datos (SGBD) regula el tipo de acceso permitido sobre un recurso en un momento determinado, impidiendo que otras transacciones realicen operaciones incompatibles hasta que dicho bloqueo sea liberado (Fernández, s.f.).

Tipos de bloqueos

Una vez comprendido el concepto de bloqueo, es necesario analizar los diferentes tipos que pueden aplicarse dentro de un sistema de bases de datos, ya que estos dependen del tipo de operación que se desea realizar.

Entre los principales se encuentran los bloqueos compartidos (S), los cuales permiten la lectura simultánea de un recurso por múltiples transacciones; los bloqueos exclusivos (X), que permiten tanto la lectura como la escritura, impidiendo el acceso de otras transacciones; y los bloqueos de actualización (U), utilizados cuando una transacción tiene la intención de modificar un dato.

Asimismo, existen bloqueos de intención jerárquicos, como IS, IX y SIX, que permiten gestionar bloqueos en distintos niveles de granularidad dentro de la base de datos, facilitando un control más eficiente de los accesos (IBM Corporation, 2018).

Funcionamiento

El funcionamiento de los bloqueos en un sistema de gestión de bases de datos se basa en la asignación y control de accesos a los recursos durante la ejecución de transacciones. Estos bloqueos son gestionados por el sistema de gestión de bases de datos (SGBD), el cual los crea en el momento en que una transacción solicita acceder a un recurso específico.

Cuando una transacción requiere acceder a un dato, el sistema evalúa si puede conceder el bloqueo solicitado o si debe colocar la transacción en espera debido a conflictos con otros bloqueos existentes.

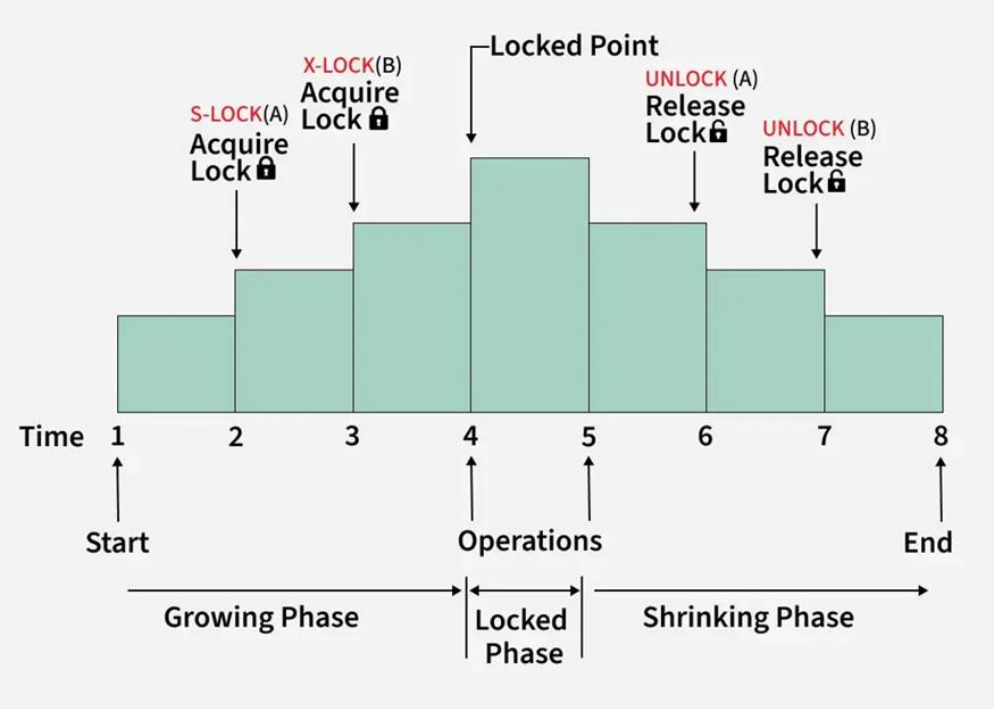
Con el fin de manejar este proceso, los bloqueos se administran mediante distintos protocolos, siendo uno de los más utilizados el protocolo de dos fases (Two-Phase Locking o 2PL). Este establece que toda transacción inicia con una fase de crecimiento, en la cual adquiere los bloqueos necesarios, seguida por una fase de encogimiento, en la que los libera. Este comportamiento garantiza la serializabilidad de las transacciones, es decir, que los resultados sean equivalentes a una ejecución secuencial (GeeksforGeeks, s.f.).

Finalmente, los bloqueos se liberan generalmente al finalizar la transacción mediante una operación de confirmación (COMMIT) o cancelación (ROLLBACK), lo que permite que otras transacciones accedan a los recursos previamente bloqueados.

Como se muestra en la Figura 1, el protocolo 2PL se divide en una fase de crecimiento, donde se adquieren los bloqueos, y una fase de encogimiento, donde estos son liberados.

Figura 1

Representación del protocolo Two-Phase Locking (2PL)



Nota. Adaptado de Two-Phase Locking Protocol, por GeeksforGeeks, s.f.

Compatibilidad de bloqueos

La compatibilidad de bloqueos permite identificar qué tipos de bloqueos pueden coexistir sobre un mismo recurso sin generar conflictos entre transacciones. Este concepto resulta fundamental para mantener un equilibrio adecuado entre la concurrencia y la consistencia de los datos.

En términos generales, los bloqueos compartidos (S) son compatibles entre sí, lo que posibilita que varias transacciones realicen operaciones de lectura de forma simultánea. No obstante, estos no son compatibles con los bloqueos exclusivos (X), ya que dichos bloqueos requieren control total sobre el recurso.

Por otro lado, los bloqueos exclusivos (X) no pueden coexistir con ningún otro tipo de bloqueo, debido a que implican operaciones de escritura que podrían comprometer la integridad de la información. En cuanto a los bloqueos de actualización (U), estos pueden coexistir con los bloqueos compartidos, pero no con otros bloqueos de actualización ni con los exclusivos, evitando así posibles conflictos durante la modificación de los datos (IBM Corporation, 2018).

Esta compatibilidad suele representarse mediante matrices que permiten visualizar de forma clara las combinaciones permitidas y no permitidas entre los distintos tipos de bloqueo.

Niveles de aislamiento

En relación con la gestión de bloqueos, los niveles de aislamiento definen el grado en que una transacción puede verse afectada por otras transacciones concurrentes. Estos niveles establecen qué tipo de anomalías pueden presentarse durante la ejecución, tales como lecturas sucias, lecturas no repetibles y lecturas fantasma.

El nivel más bajo es ,Read Uncommitted el cual permite que una transacción lea datos que aún no han sido confirmados por otra, lo que puede generar inconsistencias. Seguidamente, el nivel Read Committed impide las lecturas sucias, permitiendo únicamente acceder a datos confirmados, aunque aún pueden ocurrir lecturas no repetibles.

Por su parte, el nivel Repeatable Read garantiza que los datos leídos por una transacción no cambien durante su ejecución, evitando las lecturas no repetibles; sin embargo, todavía pueden presentarse lecturas fantasma. Finalmente, el nivel Serializable es el más estricto, ya que impide todas estas anomalías, asegurando que las transacciones se ejecuten como si fueran secuenciales (PostgreSQL Global Development Group, 2026).

En términos generales, a medida que aumenta el nivel de aislamiento, también lo hace el control sobre la consistencia de los datos; no obstante, esto puede impactar el rendimiento del sistema debido a una mayor restricción en la concurrencia.

Las diferencias entre los niveles de aislamiento pueden observarse de forma más clara en la Tabla 1, donde se resumen las anomalías permitidas y su implementación mediante bloqueos.

Tabla 1 *Comparación de niveles de aislamiento*

Nivel de aislamiento	Lecturas sucias	Lecturas no repetibles	Fantasmas	Implementación vía bloqueos
Read Uncommitted	X (permitida)	X (permitida)	X (permitida)	Transacciones que no aplican aislamiento; no adquieren bloqueos de lectura
Read Committed	X (prohibida)	X (permitida)	X (permitida)	Cada lectura adquiere bloqueo S en la fila (liberado tras la lectura)
Repeatable Read	X (prohibida)	X (prohibida)	X (permitida)	Se retienen S en todas las filas leídas hasta el commit
Serializable	X (prohibida)	X (prohibida)	X (prohibida)	Se adquieren bloqueos de rango o de tabla (bloqueos S/X amplios) hasta el commit

Nota. La “X” indica anomalías permitidas. Adaptado de Microsoft Corporation (2025) y PostgreSQL Global Development Group (2026).

Beneficios de los bloqueos

El uso de bloqueos en los sistemas de gestión de bases de datos aporta múltiples beneficios, principalmente relacionados con la integridad y consistencia de la información. Estos mecanismos permiten controlar el acceso concurrente a los

datos, evitando problemas como las actualizaciones perdidas y las lecturas de datos no confirmados.

Asimismo, los bloqueos garantizan que las transacciones se ejecuten de manera segura, manteniendo las propiedades ACID, especialmente en lo referente al aislamiento. Esto asegura que cada transacción opere sobre un estado consistente de la base de datos, incluso en entornos con múltiples usuarios accediendo simultáneamente. En conjunto, el uso adecuado de bloqueos contribuye a la estabilidad del sistema y a la confiabilidad de la información, lo cual resulta fundamental en aplicaciones críticas donde la precisión de los datos es esencial (IBM Corporation, 2018).

Desventajas de los bloqueos

No obstante, el uso de bloqueos también presenta ciertas desventajas, especialmente en entornos con alta concurrencia. La principal limitación es la contención de recursos, ya que múltiples transacciones pueden verse obligadas a esperar para acceder a un mismo dato, lo que puede disminuir el rendimiento del sistema.

Además, los bloqueos pueden dar lugar a situaciones de interbloqueo (deadlocks), en las cuales dos o más transacciones quedan en espera mutua, impidiendo el progreso de las operaciones hasta que el sistema intervenga para resolver el conflicto.

Por otra parte, la gestión de bloqueos implica un costo adicional para el sistema, tanto en términos de memoria como de procesamiento, lo que puede afectar la escalabilidad. Por ello, es necesario aplicar buenas prácticas en su uso, como mantener transacciones cortas y optimizar las consultas, con el fin de minimizar su impacto negativo (Microsoft Corporation, 2023.

Deadlocks

¿Qué es un Deadlock?

Un deadlock (o interbloqueo) es una situación en la que dos o más transacciones quedan bloqueadas indefinidamente, porque cada una espera que la otra libere un recurso que necesita para continuar. Ninguna puede avanzar, y el sistema queda en un estado de espera circular sin resolución automática.

Condiciones necesarias (Coffman, 1971)

Para que ocurra un deadlock, deben cumplirse simultáneamente las cuatro condiciones de Coffman:

1. Exclusión mutua: El recurso solo puede ser utilizado por una transacción a la vez. Si otra lo necesita, debe esperar.

2. Retención y espera (Hold and Wait): Una transacción retiene al menos un recurso ya adquirido mientras espera obtener otros recursos adicionales.

3. No apropiación (No preemption): Los recursos no pueden ser arrebatados forzosamente a una transacción; esta debe liberarlos voluntariamente.

4. Espera circular (Circular wait): Existe una cadena circular de transacciones $T_1 \rightarrow T_2 \rightarrow ... \rightarrow T_n \rightarrow T_1$, donde cada una espera un recurso retenido por la siguiente.

Detección y solución

Cuando el sistema no previene deadlocks, necesita mecanismos para detectarlos y resolverlos.

Detección: Grafo de espera (Wait-for Graph)

Se construye un grafo dirigido donde cada nodo es una transacción. Se traza una arista de $T_i \rightarrow T_j$ si T_i espera que T_j libere un recurso. Si el grafo contiene un ciclo, existe un deadlock. El sistema ejecuta periódicamente un algoritmo de detección de ciclos.

Solución: Selección de víctima (Victim Selection)

Una vez detectado el deadlock, el sistema debe romper el ciclo. Los criterios para elegir la transacción víctima incluyen:

- La transacción que lleva menos tiempo ejecutándose.
- La que ha adquirido menos recursos.
- La que ha realizado menos actualizaciones.
- La de menor prioridad según políticas del sistema.

La transacción víctima se aborta y reinicia (rollback), liberando los recursos para que las demás puedan continuar.

Prevención

La prevención busca garantizar que al menos una de las cuatro condiciones de Coffman no pueda cumplirse nunca.

Estrategia	**Condición que elimina**	**Descripción**
Adquisición de todos los recursos al inicio	Retención y espera	La transacción solicita todos los recursos que necesitará antes de ejecutar. Si no puede obtenerlos todos, no inicia.
Ordenamiento de recursos	Espera circular	Se impone un orden global sobre los recursos (ej. por ID). Las transacciones sólo pueden adquirirlos en ese orden.
Wait-Die	No apropiación (indirecto)	Las transacciones más antiguas esperan, las más nuevas son abortadas.
Wound-Wait	No apropiación (indirecto)	Transacciones más antiguas "hieren" (abortan) a las más nuevas, las más nuevas esperan.

Ventajas del manejo de deadlocks

- Continuidad del sistema: El sistema puede seguir operando sin intervención humana al detectar y resolver deadlocks automáticamente.
- Mayor concurrencia: Comparado con la prevención estricta, la detección permite mayor paralelismo porque las transacciones no necesitan reservar todos los recursos al inicio.
- Correctitud garantizada: Los datos permanecen consistentes: la transacción víctima se revierte completamente antes de reiniciarse.
- Flexibilidad: Los algoritmos de selección de víctima se pueden ajustar según las prioridades del negocio (ej: proteger transacciones críticas).
- Uso eficiente de recursos: A diferencia de la prevención, no se retienen recursos innecesariamente durante largos períodos.

Desventajas del manejo de deadlocks

- Overhead computacional: La detección requiere ejecutar algoritmos de ciclo periódicamente sobre el grafo de espera, lo que consume CPU y memoria.
- Latencia de detección: Existe un intervalo entre que ocurre el deadlock y que el sistema lo detecta; durante ese tiempo las transacciones están bloqueadas sin avanzar.
- Pérdida de trabajo: La transacción víctima pierde todo el trabajo realizado y debe reiniciarse desde cero (starvation si siempre se elige la misma).
- Complejidad de implementación: Mantener y actualizar el grafo de espera en tiempo real es complejo, especialmente en sistemas distribuidos.
- Posibilidad de inanición (Starvation): Si el criterio de selección de víctima siempre elige la misma transacción, esta nunca termina. Se requieren mecanismos adicionales para evitarlo (ej. incluir la antigüedad como factor de selección).

Conclusión

A lo largo de esta investigación se analizaron los principales mecanismos de control de concurrencia en bases de datos: las transacciones, los bloqueos y los deadlocks. La profundización en cada uno de estos temas permitió comprender cómo los sistemas gestores garantizan la integridad y consistencia de la información en entornos multiusuario.

Se determinó que la concurrencia, si bien mejora el rendimiento al permitir la ejecución paralela de transacciones, introduce riesgos como las actualizaciones perdidas, las lecturas sucias y las lecturas fantasma cuando no se gestiona adecuadamente. Las propiedades ACID constituyen el fundamento esencial para que las operaciones se ejecuten de forma confiable incluso ante accesos simultáneos.

En cuanto a los bloqueos, se comprendió que los distintos tipos existentes, junto con el protocolo de dos fases (2PL) y los niveles de aislamiento, conforman una capa de control que equilibra la concurrencia con la protección de los datos. No obstante, su uso inadecuado puede derivar en contención de recursos, lo que refuerza la importancia de mantener transacciones cortas y consultas optimizadas.

Respecto a los deadlocks, se identificó que las cuatro condiciones de Coffman explican su origen y que existen estrategias eficaces tanto para detectarlos, mediante grafos de espera, como para prevenirlos, a través del ordenamiento de recursos o los esquemas Wait-Die y Wound-Wait.

En conclusión, el control de concurrencia es un conjunto interconectado de mecanismos indispensable en cualquier sistema de bases de datos moderno, cuya correcta aplicación garantiza la integridad, disponibilidad y confiabilidad de la información en entornos críticos.

Bibliografía

Coffman, E. G., Elphick, M., & Shoshani, A. (1971). *System deadlocks*. ACM Computing Surveys, 3(2), 67–78. https://doi.org/10.1145/356586.356588

Connolly, T., & Begg, C. (2015). *Sistemas de bases de datos: un enfoque práctico para diseño, implementación y gestión* (6.ª ed.). Pearson.

Date, C. J. (2004). *An Introduction to Database Systems* (8th ed.). Pearson Addison Wesley.

Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentos de sistemas de bases de datos* (7.ª ed.). Pearson.

GeeksforGeeks. (s.f.). *Two phase locking protocol*. https://www.geeksforgeeks.org/dbms/two-phase-locking-protocol/

IBM Corporation. (2018). *Db2 13 Performance: Simultaneidad y bloqueos*. IBM. https://www.ibm.com/docs/es/db2-for-zos/13.0.0?topic=concurrency-locks

Microsoft Corporation. (2023). *SQL Server deadlocks guide*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/relational-databases/sql-server-deadlocks-guide

Microsoft Corporation. (2025). *Transaction isolation levels (ODBC)*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/odbc/reference/develop-app/transaction-isolation-levels

PostgreSQL Global Development Group. (2026). *Transaction isolation*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/transaction-iso.html

Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2011). *Fundamentos de bases de datos* (6.ª ed.). McGraw-Hill